

Contexto breve:

Reconstruir desde cero una aplicación llamada **Gastos Distribuidos** (antes en Django con templates gigantes). Debe ser una aplicación web multi-tenant para gestión de adquisiciones y distribución de gastos, procesar CFDI 4.0 (XML), generar PDFs, y soportar un flujo de compras entre áreas, proveedores, tesorería y almacén. Evitar usar plantillas HTML largas; usar API (Django REST Framework) + SPA moderna (React + TypeScript). Diseño y código listos para producción: Docker, Postgres, Celery, Redis, S3-compatible storage.

Requerimientos técnicos (especificar en cada archivo generado)

Stack principal

- Backend: Python 3.11+, Django 4.x, Django REST Framework.
- Autenticación: JWT (djangorestframework-simplejwt) + RBAC.
- Multi-tenant: PostgreSQL con `django-tenants` o `django-tenant-schemas` (usar esquema por tenant) — opción preferida `django-tenants`.
- Base de datos: PostgreSQL (no SQLite en producción).
- Colas y tareas: Celery + Redis (o RabbitMQ) para procesamiento asíncrono (PDF, parseo CFDI, envío emails).
- Almacenamiento: S3 (aws-sdk / django-storages) para facturas, evidencias fotográficas y PDFs.
- Frontend: React + TypeScript + Vite, TailwindCSS para estilo. Componentes modulares y accesibles.
- PDFs: WeasyPrint o ReportLab (WeasyPrint recomendado para convertir HTML a PDF).
- XML parsing: `lxml` y `xmldict` con validaciones específicas para CFDI 4.0.
- Tests: pytest + pytest-django + factory-boy.
- CI/CD: GitHub Actions (lint, tests, build, push Docker image).
- Infra local: Docker + docker-compose para dev; manifiestos k8s opcional para producción.

Calidad y seguridad

- Usar linters (flake8/ruff), black/isort, pre-commit hooks.
 - Validaciones OWASP básicas (rate-limit, input sanitization, CORS sec, helmet en frontend).
 - Escapar/validar archivos XML antes de procesar.
 - Registros estructurados (JSON) y trazabilidad (request id).
-

Arquitectura propuesta (alta nivel)

- **API Gateway / Backend (Django + DRF):**
 - Apps por dominio: `tenants`, `accounts`, `companies`, `areas`, `procurement`, `quotations`, `orders`, `inventory`, `invoices`, `documents` (pdfs), `notifications`.
 - Cada app con: models, serializers, views (ViewSets), urls, permissions, tests.
 - Servicios auxiliares (services/*.py) para lógica compleja (p. ej. procesamiento CFDI).
 - Tasks Celery en `procurement.tasks` y `invoices.tasks`.
 - **Frontend SPA (React + TypeScript):**
 - Pages: Login, Dashboard, Empresas, Areas, SolicitudMaterial (crear/editar), Flujo de Cotizaciones, Autorizaciones, OrdenCompra, Proveedor Portal, Almacén (recepciones), Distribución, Visualizador de PDFs.
 - Componentes: Table, Form (react-hook-form + validation), FileUploader (chunked), Modal, EvidenceGallery, Notifications.
 - API client: axios con interceptores para refresh token, requestId y retries.
 - **Integraciones:**
 - S3 (media), SMTP (envío de notificaciones), optionally Signer/Certificado para validar CFDI si aplica.
-

Modelos (esqueleto con campos recomendados — generar migrations)

Incluye estos modelos con tipos y relaciones (añadir campos auditables: created_by, created_at, updated_at, tenant):

Tenancy

- Tenant (subdominio/schema, nombre, RFC, settings, is_active)

Auth / Accounts

- User (AbstractUser): username, email, full_name, role(FK Role), tenant (FK Tenant/tenant-schemas), is_active.
- Role: name (Admin, Tesorería, Adquisiciones, Almacén, Área, Proveedor), permissions (JSON / ManyToMany).

Companies & Areas

- SolicitudGubernamental: datos solicitante, RFC, estado (pendiente/aprobada/rechazada), attachments
- AntePubli (Company): rfc, razon_social, address, contact_info, tenant link
- Area: name, company, manager(FK User), code

Procurement

- SolicitudMaterial: id, area (FK Area), created_by, estado (borrador/enviado/aceptado), fecha_solicitud, descripcion, total_estimated
- DetalleMaterial: solicitud (FK), concepto, cantidad (Decimal), unidad, COG (FK Cog), notas
- Cog: codigo, descripcion

Quotations

- CotizacionMaterial: solicitud (FK), proveedor (FK Proveedor), fecha, total
- CotizacionDetalle: cotizacion (FK), detalle_material (FK DetalleMaterial or free-form), precio_unitario, subtotal
- CotizacionSolicitud: m2m linking cotizaciones y solicitudes

Authorizations & Orders

- SolicitudAutorizacion: solicitud (FK SolicitudMaterial), solicitante, estado
- AutorizacionPresupuestal: solicitud_autorizacion (FK), aprobado_por (FK), monto_autorizado, fecha_aprobacion

- OrdenCompra: proveedor (FK), autorizacion (FK), fecha_emision, estado, referencia_externa
- DetalleOrden: orden (FK), material (FK DetalleMaterial), cantidad, precio_unitario

Invoices & Inventory

- Factura: prov (FK Proveedor), xml_file (FileField), uuid_cfdi, folio, fecha, subtotal, iva, total, status (procesada/error), parsed_json (JSONField)
- FacturaDetalle: factura (FK), concepto, cantidad, precio_unitario, impuestos
- EntregaBienes: orden (FK OrdenCompra), factura (FK Factura), fecha_recepcion, recibido_por (FK User), notas
- EntregaDetalle: entrega (FK), detalle_orden (FK), cantidad_recibida, photos (M2M to Media)
- SalidaBienes: almacen (FK Area), fecha, responsable, destino_area (FK Area), referencia
- SalidaDetalle: salida (FK), material, cantidad

Media / Documents

- Media: file, owner, content_type, created_at, metadata (JSON)
- PDFDocument: related_model (Generic FK), pdf_file, generated_by_task, timestamp

Endpoints REST clave (especificar path, método, descripción)

Autenticación

- `POST /api/auth/token/` → login (JWT)
- `POST /api/auth/token/refresh/` → refresh token

Tenancy & Empresas

- `POST /api/tenants/solicitud/` → crear SolicitudGubernamental
- `GET /api/tenants/` (Admin) → listar empresas

Areas & Personal

- `GET /api/areas/` `POST /api/areas/`

- `GET /api/personal/` `POST /api/personal/`

Solicitudes & Materiales

- `POST /api/solicitudes/` → crear solicitud material con detalles
- `GET /api/solicitudes/{id}/` (detalle + links a cotizaciones)

Cotizaciones

- `POST /api/cotizaciones/{solicitud_id}/` → proveedor crea cotización
- `GET /api/cotizaciones/?solicitud={id}`

Autorizaciones

- `POST /api/autorizaciones/` → tesorería aprueba
- `GET /api/autorizaciones/{id}/pdf/` → descargar autorización en PDF

Órdenes de compra

- `POST /api/ordenes/` → generar orden de compra desde cotización seleccionada
- `POST /api/ordenes/{id}/send/` → envía al proveedor (notificación + email)

Facturas / CFDI

- `POST /api/facturas/upload/` → upload XML (multipart). Respuesta inmediata: registro en BD y task async para parseo.
- `GET /api/facturas/{id}/parsed/` → ver JSON parseado
- `POST /api/facturas/{id}/distribute/` → tarea para mapear conceptos a solicitudes/areas (ejecución async)

Almacén / Entregas

- `POST /api/entregas/{orden_id}/` → registrar recepción, subir evidencia (fotos)
- `POST /api/salidas/` → generar salida de bienes hacia área destino

Documentos & PDFs

- `GET /api/documents/{id}/pdf/` → descargar pdf generado
- `POST /api/documents/generate/` → task que genera PDF (WeasyPrint) en background

Proveedores (portal reducido)

- `POST /api/proveedores/signup/` → registro proveedor (pending)

- `GET /api/proveedores/{id}/orders/` → proveedor ve sus ordenes

Extras

- `GET /api/reports/monthly-spend/` → reportes agregados por COG, area, proveedor (paginado)
- `GET /api/alerts/` → notificaciones y actividades pendientes

Flujos asíncronos y tareas (Celery)

- **Parseo CFDI:** cuando se sube XML → valida esquema, extrae UUID, rfc emisor/receptor, conceptos, impuestos. Guardar JSON en `Factura.parsed_json`. Si falla, marcar `status=error` y notificar a tesorería.
- **Generación PDF:** plantillas HTML (componentes pequeños) renderizados server-side en WeasyPrint en background.
- **Distribución automática:** tarea que asociará conceptos de factura con solicitudes/entregas y generará salidas parciales.
- **Notificaciones:** envío de correos y push notifications (webhook) asíncrono.

Frontend: estructura mínima a generar

- `src/`
 - `pages/` (Login, Dashboard, Solicitudes, Cotizaciones, Autorizaciones, OrdenCompra, Almacen, Proveedor)
 - `components/` (DataTable, FormFields, FileUploader, PDFViewer, ImageEvidence, RoleGuard)
 - `services/api.ts` (axios wrapper)
 - `hooks/` (useAuth, useFetchPaged)
 - `stores/` (zustand o React Context para auth + tenant)
 - `routes.tsx` (protected routes por rol)
 - UI: Tailwind + Headless UI o Radix para accesibilidad

UX notes:

- Validaciones de formularios en cliente (react-hook-form + zod)
- Subida de archivos con progreso y reintentos

- Paginación server-side y filtros para listados grandes
 - Viewer embebido para XML/PDF
-

Migrations, Fixtures y Seeds

- Comandos `manage.py` :
 - `seed_demo_data` (crear tenant demo, admin, áreas, usuarios, proveedores, solicitudes y cotizaciones).
 - `import_cfdi --file path` (para testing local del parser).
-

Tests y criterios de aceptación

- Cobertura mínima de backend: endpoints críticos (auth, upload factura, generar orden, distribuir) con tests de integración.
- Tests unitarios para services de parseo CFDI y generación de PDF.
- Tests frontend: pruebas E2E (Cypress/Playwright) para flujo de solicitud → cotización → orden → recepción.

Criterios de aceptación al terminar:

1. Backend API con Postgres + migrations funcionando en Docker.
2. Endpoint upload XML que parsea y guarda datos en BD. (Celery procesa el XML).
3. Generación de PDF en background y endpoint para descarga.
4. Frontend SPA funcional con login, crear solicitud y ver cotizaciones.
5. CI: GitHub Actions que corre linter y tests al push.
6. Documentación OpenAPI/Swagger y README con pasos de puesta en marcha.

Modelo de Roles (CRÍTICO)

El sistema **DEBE implementar control de acceso basado en roles**, tanto en backend (permissions) como en frontend (rutas, UI y acciones).



ROL: ADMINISTRADOR (Admin Global)

Qué ve

- Solicitudes de registro de empresas:
 - Pendientes
 - Aprobadas
 - Rechazadas (con historial)
- Dashboard global:
 - Total de solicitudes
 - Aprobadas / Pendientes / Rechazadas
- Listado de empresas activas
- Gestión interna de cada empresa:
 - Áreas
 - Personal
 - Roles

Qué puede hacer

- Aprobar solicitudes de registro (crea empresa/tenant)
- Rechazar solicitudes
- Crear áreas por empresa
- Crear usuarios (personal)
- Asignar roles
- Activar / desactivar empresas

Importancia

El Admin es **el guardián del sistema**.

Sin su aprobación, ninguna empresa existe.

Es el **único rol con control estructural** del sistema.



ROL: TESORERÍA

Qué ve

- Dashboard financiero:
 - Total de solicitudes
 - Pendientes de autorización
 - Autorizaciones aprobadas
- Solicitudes de materiales (todas las áreas)
- Cotizaciones comparadas por proveedor
- Salidas de bienes asignadas a su área
- Vista de comprobación de facturas CFDI

Qué puede hacer

- Revisar cotizaciones
- Seleccionar proveedor ganador
- Aprobar o rechazar suficiencia presupuestaria
- Generar autorización presupuestal
- Enviar autorización a Adquisiciones
- Subir XML CFDI 4.0
- Validar y procesar facturas
- Distribuir gastos entre áreas

Importancia

Tesorería es **el corazón financiero**.

Nada se compra sin su visto bueno.

Valida dinero, proveedores y facturación fiscal.



ROL: ADQUISICIONES

Qué ve

- Solicitudes de materiales aprobadas
- Materiales agrupados por COG

- Estado de cotizaciones (mínimo 3)
- Proveedores registrados
- Autorizaciones recibidas
- Órdenes de compra generadas

Qué puede hacer

- Enviar solicitudes de cotización
- Comparar precios
- Enviar cotizaciones a Tesorería
- Generar solicitudes de suficiencia
- Crear órdenes de compra
- Enviar órdenes al proveedor
- Registrar proveedores

Importancia

Adquisiciones es **el motor del proceso de compra**.

Garantiza transparencia, competencia y documentación correcta.

ROL: ALMACÉN

Qué ve

- Órdenes de compra pendientes
- Detalle de pedidos
- Información de factura
- Entregas registradas
- Salidas de bienes por área

Qué puede hacer

- Registrar recepción de bienes
- Confirmar ítems recibidos
- Subir evidencia fotográfica

- Documentar entregas incompletas
- Generar salidas de bienes
- Enviar salidas a las áreas

Importancia

Almacén es **el control físico** del sistema.

Sin él, no hay trazabilidad real de los bienes.



ROL: ÁREAS / DEPARTAMENTOS

Qué ve

- Sus solicitudes de materiales
- Estado de cada solicitud
- Salidas de bienes asignadas
- Formularios de confirmación

Qué puede hacer

- Crear solicitudes de materiales
- Ver progreso del trámite
- Confirmar recepción
- Agregar observaciones
- Subir evidencia

Importancia

Las áreas son **el origen del flujo**.

Sin solicitudes, no existe el sistema.



ROL: PROVEEDOR (Externo)

Qué ve

- Cotizaciones recibidas

- Formularios de precios
- Órdenes de compra asignadas
- PDFs descargables
- Configuración de perfil

Qué puede hacer

- Ingresar precios
- Enviar cotizaciones
- Ver órdenes de compra
- Descargar documentos
- Actualizar información

Importancia

El proveedor es **el actor externo**.

Compite, cotiza y entrega.

Sin proveedores, no hay compras.

Reglas de acceso (obligatorias)

- Cada endpoint debe validar:
 - Tenant
 - Rol
 - Estado del flujo
 - El frontend **no debe renderizar** acciones no permitidas por rol
 - Usar permisos explícitos, no ifs sueltos
-

Flujo del sistema (resumido)

1. Admin aprueba empresa
2. Área crea solicitud
3. Adquisiciones cotiza

4. Tesorería autoriza
 5. Adquisiciones genera OC
 6. Proveedor entrega
 7. Almacén recibe y distribuye
 8. Área confirma
 9. Ciclo cerrado con evidencia
-

Reglas de desarrollo

- Nada de templates HTML gigantes
 - Toda lógica de negocio va en `services/`
 - Views solo orquestan
 - PDFs se generan en background
 - CFDI se procesa async
 - Todo auditable
 - Todo testeado
-

Entregables mínimos

1. Backend con roles y permisos funcionales
2. Frontend con rutas protegidas por rol
3. Upload y parseo CFDI real
4. PDFs generados automáticamente
5. Docker + CI
6. Swagger documentado
7. README claro para otro humano